ECE 422

Secure File System

Shaheer Khan, Andy Charlton, Abs Salim, Julian Jedrych

March 24, 2026

Abstract

This project is a Secure File System (SFS) that lets users store files on a server without giving the server access to the file contents. The system is written in C and uses a client-server design. The client connects to the server over TLS, sends requests using a simple HTTP-style API, and handles important cryptographic work such as key creation, signing, encryption, and decryption. The server manages users, sessions, permissions, file metadata, and group membership. File contents are stored on disk, while metadata is stored in SQLite. The design supports multiple users, groups, directories, and owner/group/other permissions. Login is based on a challenge-response signature flow using public keys, and the server returns a session token after successful login.

Introduction

Storing files on a remote server is useful, but it creates a trust problem. If the server stores sensitive data in plain text, anyone with server access may be able to read private information or change it. The goal of this project is to build a Secure File System that reduces that risk. In this system, the client and server communicate over TLS, the client prepares or verifies protected data, and the server stores encrypted file data and metadata separately. The system supports users, groups, directories, and Unix-style permissions so files can be shared in a controlled way. It also uses public-key signatures during login instead of sending a plain-text password to the server. This report describes the current design of the project and explains how the main parts work together.

Detailed Design

Programming Language: C

Our Secure File System will be implemented in the C programming language. This decision is driven by two primary considerations. First, C has very strong performance and low-level control over system resources. For a file system, using C directly minimizes runtime overhead and allows for optimized cryptographic and file I/O operations. Second, C is the traditional language for systems programming, making it a strong choice for this project. This ensures that our SFS can efficiently map its internal concepts onto the underlying OS storage mechanisms.

Cryptographic Libraries

In this project, we will use two libraries for cryptographic operations, selecting the best tool for each specific task. For the majority of cryptographic requirements, including

authenticated encryption and key management, we will use Libsodium. Libsodium was chosen for its ease of use and safety. It provides modern cryptographic primitives with a simplified, high-level API. This abstraction reduces the risk of implementation errors and allows us to concentrate on our SFS architecture rather than low-level cryptographic details. As well, Libsodium includes an implementation of the AEGIS-256 algorithm, which we have selected for file encryption, and provides all necessary tools for our security model, such as Ed25519 for signatures and constant-time comparison functions.

The only exception to the use of Libsodium is for Transport Layer Security (TLS). For this, we will use OpenSSL. To mitigate the complexity of OpenSSL, its use will be confined to implementing the TLS protocol for network communication. All other cryptographic operations will be managed using Libsodium.

Encryption Strategies

The system protects data in two places: while it is moving across the network and while it is stored on the server. For network traffic, the client and server communicate over TLS. This protects requests and responses from being read or changed in transit. For stored data, files are encrypted before or during storage using per-file keys. The client code includes functions for generating file keys, wrapping keys for users or groups, and encrypting and decrypting file contents. The design also supports wrapped file keys for different access levels: owner, group, and other. These wrapped keys are stored in metadata so the correct user can later recover access. The login flow also uses signatures. The server creates a random challenge, the client signs it using the user's signing key, and the server verifies the signature using the stored public signing key. This proves the user has the correct private key without sending that private key to the server or storing a password on the server.

User Authentication and Authorization

User registration stores a username together with the user's public encryption key and public signing key. During login, the client first sends a username. The server responds with a random challenge. The client signs that challenge and sends the signature back. If the signature matches the stored public signing key for that user, the server creates a session token. After login, the session token is used to authorize later requests. The server keeps session information in memory, including the user ID, username, token, and expiry time. Authorization for files follows a Unix-style model with owner, group, and other permission bits. The server checks these permissions before allowing reads, writes, moves, deletes, or directory operations.

Data Storage Architecture

The system uses two storage layers. SQLite stores metadata, and the server's file storage directory holds file contents. The database stores users, groups, group memberships, file paths, permissions, object type, wrapped file keys, and timestamps. The database does not store the actual plaintext file contents. The file data itself is stored under the server storage root

on disk. This separates content storage from metadata management. The server can quickly check permissions and locate files using SQLite, while the filesystem holds the actual stored objects. The design also supports group-based sharing. Group membership records include a wrapped group key, and file metadata can include wrapped file keys for the owner, a group, and others. This makes it possible to control access without exposing file contents directly in the database.

Networking and User Interface

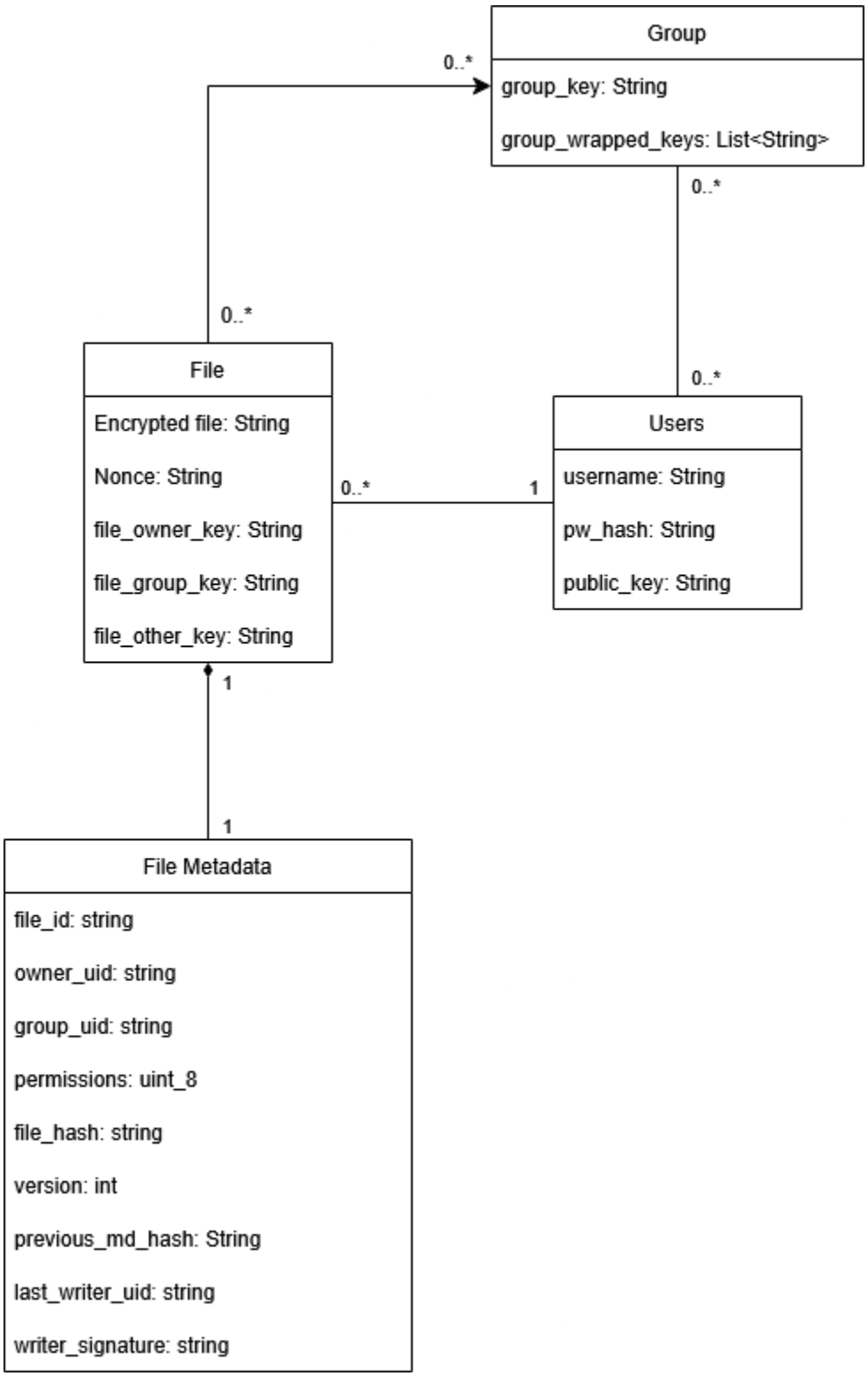
The client and server communicate using a simple HTTP-style request and response layer over TLS. The server routes requests by method and path, such as /auth/login, /auth/register, /files, /directories, and /groups. JSON is used for many request and response bodies, especially for authentication, group management, and metadata-related operations. Raw file data is sent separately when reading or writing file contents.

The user interacts with the system through a command-line interface. The codebase includes login and logout support and is structured for common file-system actions such as listing files, creating directories, moving files, and removing files. This keeps the interface simple and makes the project easier to test and extend.

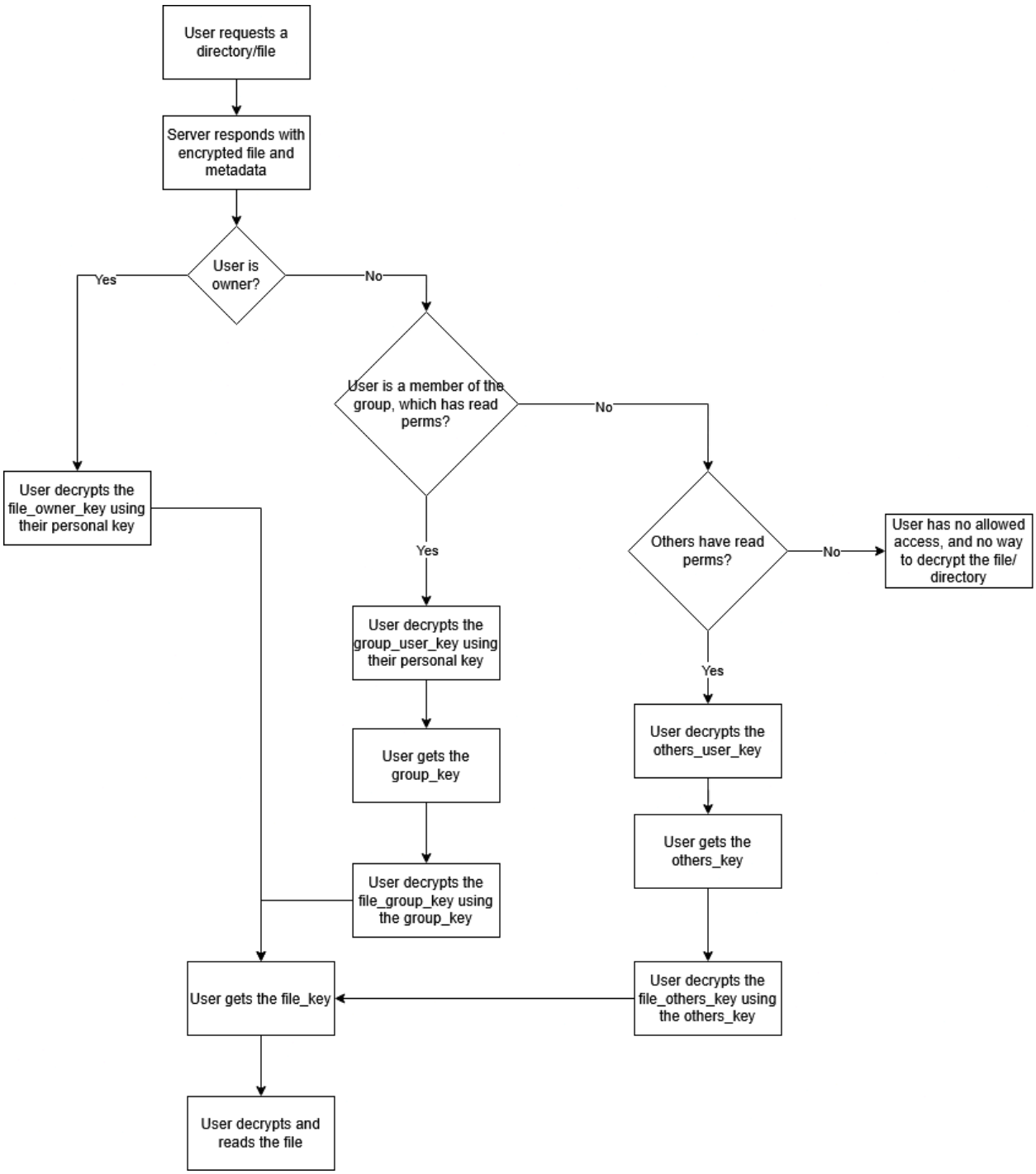
Development Methodology

The project will be developed using an Agile methodology with iterative sprints. This approach is well-suited to a fast-turnaround project with many well-defined requirements. By breaking the work into smaller, manageable increments, we can deliver functional software quickly. The iterative nature of Agile provides the flexibility to adapt to technical challenges and refine our design as we learn, while continuous integration within short sprints ensures that components work together correctly and reduces integration risks. Progress will be tracked using user stories on our project board, providing clear visibility into the state of development.

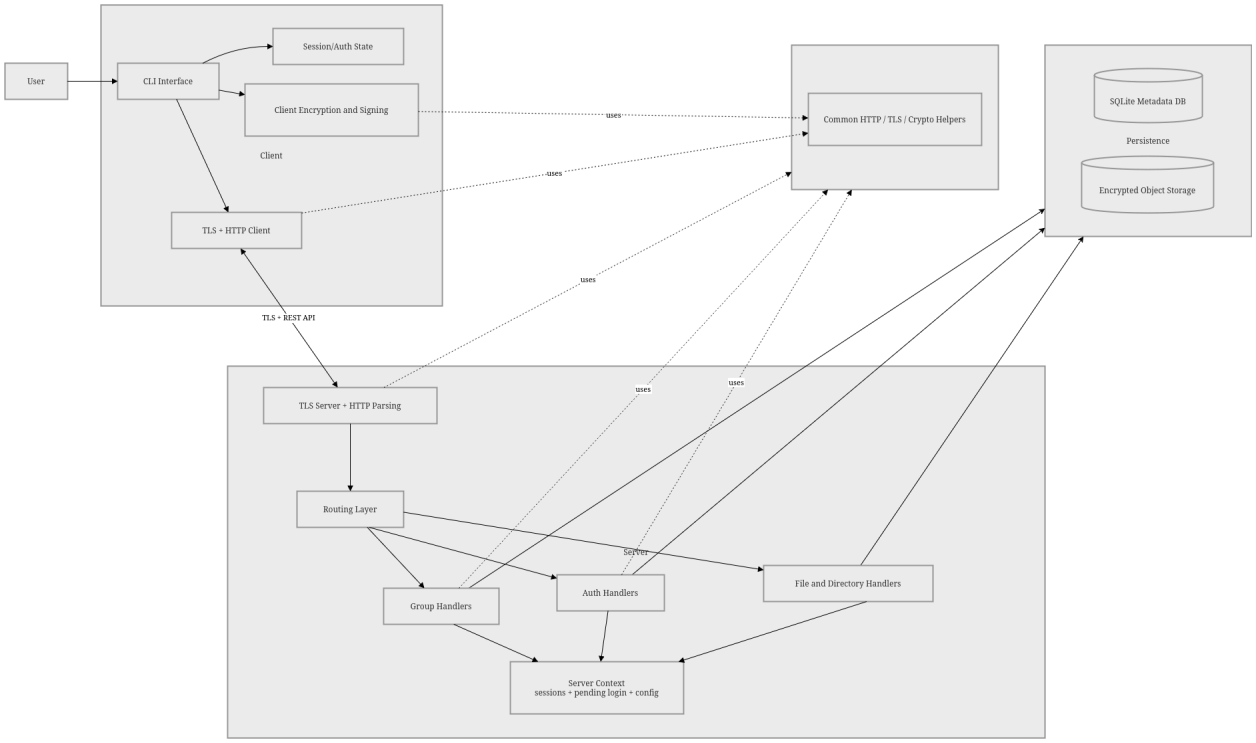
Class Diagram



Sequence Diagram



High Level Architecture Diagram



User Stories

Authentication & Secure Communication

1. As a user, I want to log into the SFS using my username and password so that only authorized users can access the system.

2. As a user, I want communication between my client and the SFS server to be encrypted so that my credentials and file data cannot be intercepted.

3. As a system administrator, I want user credentials to be securely hashed and stored so that stolen password databases cannot easily be exploited.

File Operations

4. As a user, I want to create files so that I can store data within the secure file system.

5. As a user, I want to read files so that I can access my stored information.

6. As a user, I want to modify files so that I can update the information stored in them.

7. As a user, I want to delete files so that I can remove data that is no longer needed.

8. As a user, I want to rename files so that I can better organize my data.

Directories

9. As a user, I want to create directories so that I can organize files in a structured way.

10. As a user, I want to navigate directories similar to a Unix filesystem so that the system is intuitive to use.

11. As a user, I want to have a home directory automatically created for me so that I have a personal space for my files.

Permissions & Sharing

12. As a user, I want to assign owner, group, and other permissions to files and directories so that I can control who can access them.

13. As a system administrator, I want to create and manage groups so that users can collaborate securely.

Encryption & Data Protection

14. As a system, I want file names to be encrypted on disk so that external users cannot determine the meaning of stored files.

15. As a system, I want file contents to be encrypted before being stored on the server so that external users cannot read sensitive data.

16. As a system, I want to verify the integrity of files and directories whenever they are accessed so that any unauthorized modifications can be detected immediately.

17. As a user, I want to be notified if a file fails an integrity check when I attempt to access it so that I know the data may have been corrupted or tampered with.

System Operation

18. As a user, I want to interact with the secure file system through a command-line interface similar to a Linux terminal so that file operations are familiar and efficient.

19. As a developer, I want the system to store only encrypted file names and contents on the untrusted server so that external users cannot read or understand stored data.

Deployment/Setup

The Secure File System is built and run locally. To set it up, you first need the required build tools and libraries, then you build the project with CMake, create the needed certificate and storage files, start the server with the correct environment variables, and finally run the client.

On Ubuntu or Debian, install the required tools and libraries with:

```
sudo apt update
sudo apt install -y build-essential cmake pkg-config libsodium-dev libssl-dev libsqlite3-dev
libcjson-dev libllhttp-dev
```

After installing dependencies, build the project from the root directory:

```
mkdir -p build
cd build
cmake ..
cmake --build .
```

This creates the main programs:

```
build/client/client
build/server/server
build/server/seed_user
```

Next, create the folders used by the server and client:

```
mkdir -p server/deploy/storage/sfs_storage
mkdir -p server/deploy/storage/sqlite_data
mkdir -p server/deploy/secrets
mkdir -p client/certs
```

The system expects the following certificate files:

```
server/deploy/secrets/server-cert.pem
server/deploy/secrets/server-key.pem
client/certs/ca-cert.pem
```

Before starting the server, set these environment variables.

Required server variables:

SERVER_CERT = path to the server certificate file

SERVER_KEY = path to the server private key file
PORT = port number the server listens on
Optional server variables:

DB_PATH = path to the SQLite database file
Default: server/deploy/storage/sqlite_data/sfs.db
DB_SCHEMA = path to the SQL schema file
Default: server/db/init/001-schema.sql
STORAGE_ROOT = path to the file storage folder
Default: server/deploy/storage/sfs_storage
SESSION_TTL_SECONDS = session timeout in seconds
Default: 3600
Example server setup and run command:

```
export SERVER_CERT=server/deploy/secrets/server-cert.pem
export SERVER_KEY=server/deploy/secrets/server-key.pem
export PORT=25565
export DB_PATH=server/deploy/storage/sqlite_data/sfs.db
export DB_SCHEMA=server/db/init/001-schema.sql
export STORAGE_ROOT=server/deploy/storage/sfs_storage
export SESSION_TTL_SECONDS=3600

./build/server/server
```

Before starting the client, set these environment variables:

CA_CERT = path to the CA certificate file
SERVER_ADDR = server IP address or hostname
SERVER_PORT = server port number
Example client setup and run command:

```
export CA_CERT=client/certs/ca-cert.pem
export SERVER_ADDR=127.0.0.1
export SERVER_PORT=25565

./build/client/client
```

The TLS certificate must match the hostname or IP address used by SERVER_ADDR. For example, if the client connects to 127.0.0.1, the certificate should be valid for 127.0.0.1. Once the server is running and the client connects successfully, the user can register, log in, and use the command-line interface.

User Guide

The Secure File System is used through the client program in the terminal. Before starting, make sure the server is running and the client environment variables are set correctly. Then launch the client with:

./build/client/client

When the client starts, it shows a simple main menu with three options:

- `login`
- `register`
- `exit`

If the user is new, they should choose “register” first. The client will ask for a username and password. These are used on the client side to generate the user’s encryption and signing key pairs. The client then sends the username and public keys to the server. If registration succeeds, the account is created and can be used to log in.

If the user already has an account, they choose “login”. The client asks for the username and password. The server responds with a login challenge, and the client signs that challenge using the signing key derived from the username and password. If the signature is verified successfully, the server returns a session token and the user enters the command interface.

After login, the user works inside the CLI. The prompt shows the username and a placeholder path format. From there, the user can enter supported commands. Based on the current design and code structure, the intended user actions include:

- `login` to sign in from the main menu
- `register` to create a new account
- `logout` to end the current session
- `ls` to list files in a directory
- `cd` to change directories
- `mkdir` to create a directory
- `create` to create a new file
- `read` to read a file
- `write` to write or update file contents
- `rm` to remove a file
- `mv` to move or rename a file
- `chmod` to change file permissions

At the moment, `logout` is clearly implemented in the interactive client, while the rest of the file commands are part of the server API design and system structure. The project is organized so that these commands map to file and directory operations on the server.

A typical session would look like this:

1. Start the client.
2. Choose `register` if creating a new account.
3. Choose `login` and enter the same username and password.
4. After successful login, use file-system commands in the CLI.
5. When finished, type `logout`.
6. Back at the main menu, type `exit` to close the client.

The server uses the session token created during login to identify the user on later requests. This means the user does not need to log in again before every action. The server checks permissions before allowing access to files or directories. These permissions follow the owner, group, and other model, so access depends on who owns the file, what group it belongs to, and what permission bits are set.

The system also supports group-based sharing. A user may be added to a group, and files can be associated with that group. This allows multiple users to access shared files when they have the proper permissions. Group membership and file metadata are managed by the server, while the actual file data is kept separately in storage.

If login fails, the most common causes are an incorrect username, an incorrect password, or a mismatch between the client keys and the public keys stored on the server. If the client cannot connect at all, the user should check that the server is running, that the certificate paths are correct, and that `SERVER_ADDR`, `SERVER_PORT`, and `CA_CERT` are set properly.

In normal use, the user experience is meant to be simple: connect to the server, log in, work with files through basic commands, and log out when done. The security steps happen mostly in the background, while the interface stays similar to a basic command-line file tool.

Conclusion

This project shows how a Secure File System can be built using a simple client-server design in C. The final system supports secure communication, user registration and login, session-based access, file and directory operations, group management, and permission checks. It also separates stored file contents from metadata by keeping metadata in SQLite and file data in server storage.

The project meets its main goal of allowing users to store and share files without fully trusting the server with their data. While the system can still be improved and expanded, the current design provides a clear foundation for secure storage, controlled sharing, and protected communication.